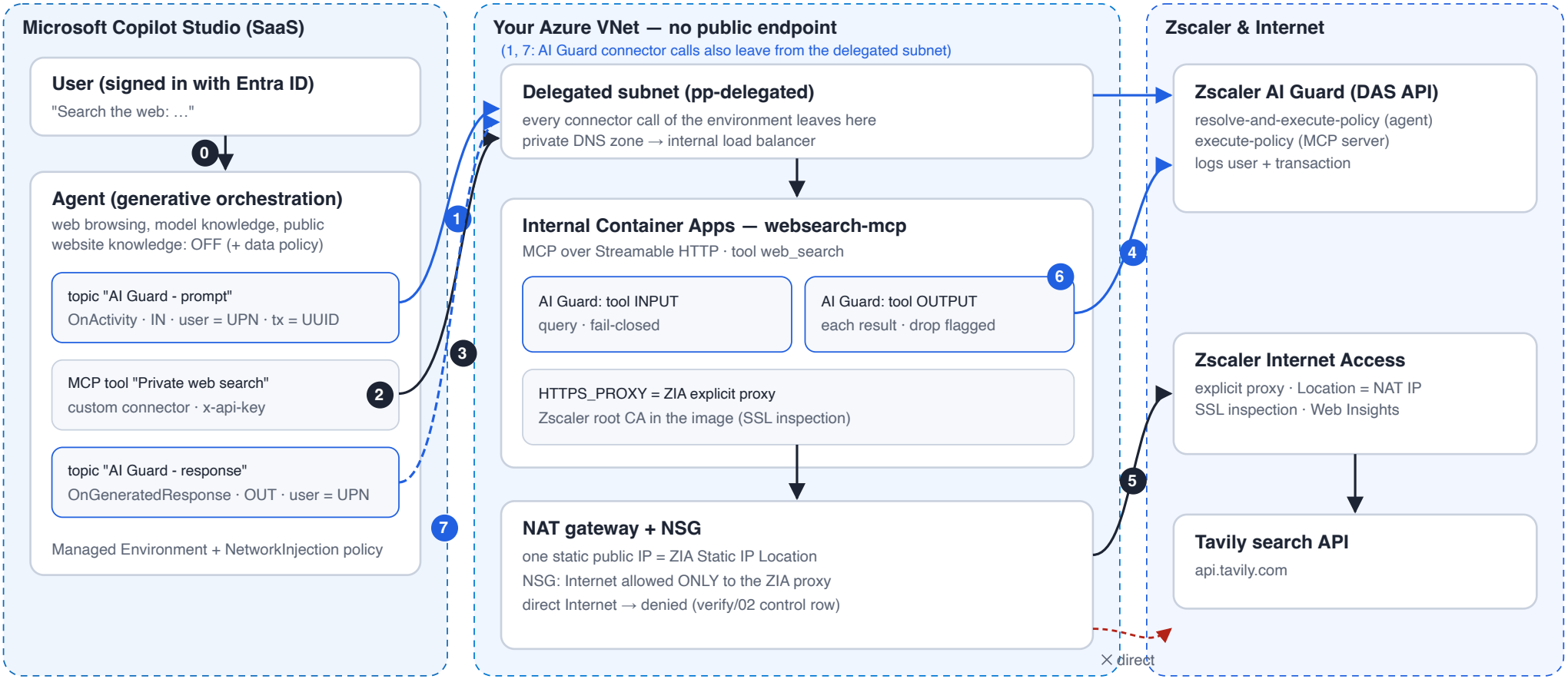
Copilot Studio agent: private MCP server, Zscaler ZIA egress, Zscaler AI Guard

One controlled path to the Internet — every prompt, tool call and answer checked by AI Guard.

Personal work by Henry Shen — not an official Zscaler solution, not supported by Zscaler, for educational purposes only. See LICENSE and NOTICE.md.



The flow, step by step

0. **Sign-in.** The agent requires Microsoft authentication, so Copilot Studio knows the user (`System.User.PrincipalName`). Anonymous channels are refused.
1. **Prompt check.** Topic *AI Guard - prompt* runs on every message *before* generative orchestration and calls AI Guard `resolve-and-execute-policy` directly through a custom connector: `direction IN`, `user` = UPN, `transactionId` = a new UUID. `BLOCK` → "Your message was blocked by Zscaler AI Guard (transaction …)" and the turn ends.
2. **Orchestration.** Generative orchestration picks the MCP tool. Built-in web search, model knowledge and public-website knowledge are off — and blocked by the environment's data policy — so Internet questions can only be answered through the tool.
3. **Private reach.** The tool is a custom connector to the MCP server's internal FQDN. Through the environment's NetworkInjection enterprise policy (Managed Environment), the call leaves from the delegated subnet; a private DNS zone resolves the name to the internal load balancer. The MCP server has no public IP and no public DNS record, and requires an API key.
4. **Tool input check.** The MCP server sends the model-chosen query to AI Guard (`execute-policy`). Blocked or no verdict → no search.
5. **Egress through ZIA.** The search goes to Tavily through the ZIA explicit proxy from the NAT gateway's static IP (a ZIA Location). The NSG denies every other Internet destination.
6. **Tool output check.** Every result is checked as model input (indirect prompt injection); flagged results are dropped before they reach the model.
7. **Response check.** Topic *AI Guard - response* holds the generated answer (`System.ContinueResponse = false`), checks it with `direction OUT` for the same user, and sends it only if it is not blocked.

What proves it (verify/)

Check	Control row	Pass row
01 private MCP	no public DNS; TCP 443 from Internet fails; /mcp without key → 401	initialize + tools/list from inside the VNet
02 ZIA-only egress	direct HTTPS from the subnet times out	same request via ZIA → 200; tool returns results
03 AI Guard API	injection → BLOCK; non-UUID tx rejected	benign → ALLOW; user + UUID accepted via a match rule
04 agent, signed in	injection → blocked by AI Guard	hi → Hello World; web question → answer with URLs
05 configuration	auth, publish status, orchestration, knowledge/browsing off, Managed Environment, VNet link, billing, data policy	
06 caller subnet	MCP server log: Copilot Studio's calls from the delegated subnet	
07 transcripts	conversation transcripts in Dataverse (Bot Transcript Viewer role)	

Design choices

Choice	Why
Classic agent via Dataverse API	agents created in the new designer (cliagent) cannot be called by the Copilot Studio API, so they cannot be verified programmatically
API key, not OAuth, on the MCP connector	OAuth token requests do not go through the VNet
AI Guard called directly by Copilot Studio	prompt and answer are checked where they exist, with the user's identity
Gate on `action = "BLOCK"`	a policy without response detectors returns `null` for OUT
Fail closed everywhere	no verdict, no answer: attribution and control are the point
Custom connectors in "General"	a Business-classified connector makes every publish fail (DLP)